Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 11

Lucrul în echipă cu Git și depanarea aplicației

Pregătim branch-ul comun al proiectului, rezolvăm un conflict provocat intenționat și găsim pas cu pas o eroare, de la browser până în jurnalul serverului.

Tema	Temele 12 și 13 din planul disciplinei (examene.pe.ub.ro): dezvoltarea în echipă și testarea/depanarea aplicațiilor
Durata	2 ore
Aveți nevoie de	aplicația din laboratoarele 07–10, contul GitHub și, pentru echipe, colegul de proiect
La final aveți	branch-ul `proiect-nume1-nume2` pe GitHub (sau branch-ul propriu, la proiectele individuale), un conflict rezolvat și o eroare găsită în jurnal

1. Obiective

La finalul laboratorului veți putea:

- crea și prelua un branch comun pentru proiectul în echipă de doi;
- lucra alternativ, cu `git pull` la început și commit-uri mici, cu mesaje clare;
- rezolva un conflict de îmbinare (*merge conflict*);
- arăta contribuția fiecăruia și găsi o eroare cu DevTools și jurnalele serverului.

2. Pe scurt: echipa și depanarea

Git păstrează istoria completă a proiectului pe fiecare calculator; GitHub este punctul comun prin care colegii își schimbă commit-urile [2]. Un **conflict** apare doar când doi oameni modifică **aceleași rânduri** ale aceluiași fișier: Git nu ghicește care variantă e corectă și vă cere vouă să decideți. Commit-urile mici și `git pull` des fac conflictele rare și ușor de rezolvat.

Proiectul în echipă se notează pentru fiecare membru, iar dovada contribuției este **istoria commit-urilor**: cine, ce și când a scris [3]. De aceea fiecare lucrează din contul lui, cu numele și e-mailul lui în Git. **Depanarea** înseamnă să găsiți metodic *unde* se oprește o cerere: în browser, în rețea sau în server [1], [4].

3. Pornirea

1. Porniți **Docker Desktop** (mesajul **Engine running**). Comenzile se scriu în terminalul editorului, în folderul `daw-template-ti-info-2026-2027`; pentru probleme, vedeți metodologia [6].
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele și verificați cu ce nume și e-mail semnați commit-urile (e-mailul trebuie să fie cel din contul GitHub):

```
git branch                 # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
git config user.name
git config user.email
```

4. Branch-ul comun al proiectului

> **Exemplu.** Echipa este formată din Ion Popescu (`popescu.ion`) și Maria Ionescu (`ionescu.maria`). Înlocuiți numele cu ale voastre. Dacă lucrați **individual**, continuați pe branch-ul propriu și faceți secțiunea 5 cu colegul de bancă, pe un branch de test.

1. **Ion** (unul singur dintre voi) creează branch-ul comun pornind din branch-ul lui și îl publică pe GitHub:

   ```
   git checkout popescu.ion
   git pull
   git checkout -b proiect-popescu-ionescu   # sau: git switch -c proiect-popescu-ionescu
   git push -u origin proiect-popescu-ionescu
   ```

2. **Maria** salvează întâi ce are în lucru pe branch-ul ei (`git add .`, `git commit`), apoi preia branch-ul comun:

   ```
   git fetch
   git checkout proiect-popescu-ionescu
   ```

3. Dacă nu ați făcut-o în laboratorul 01, setați o singură dată, pe fiecare calculator, ca `git pull` să îmbine prin *merge*:

   ```
   git config --global pull.rebase false
   ```

> **Rezultat.** Pe GitHub, în lista **Branches**, apare `proiect-popescu-ionescu`. La Maria, `git branch` arată steluța lângă branch-ul comun, iar fișierele sunt cele ale lui Ion.

4.1 Regulile lucrului în doi

- `git pull` **înainte de a începe** orice sesiune de lucru, ca să porniți de la ultima versiune.
- **Commit-uri mici**, câte unul pentru fiecare lucru terminat, cu mesaje care spun *ce* s-a schimbat: „Adaugă validarea notei la recenzii”, nu „modificări” sau „gata”.
- `git push` **la final**, ca să găsească și colegul lucrul vostru. Stabiliți cine lucrează la ce fișiere, ca să nu modificați amândoi aceleași rânduri.

5. Un conflict provocat intenționat

Lucrați amândoi pe branch-ul comun, la calculatoare diferite, și rulați întâi `git pull`. Țința: titlul din `frontend/app/src/components/ReviewSection.jsx`, rândul `<h2>Recenzii ({reviews.length})</h2>`.

1. Ion schimbă textul în `Părerile clienților`, salvează, apoi `git add .`, `git commit -m "Schimba titlul recenziilor"` și `git push`.
2. Maria, **fără** `git pull`, schimbă același rând în `Ce spun cumpărătorii`, face commit și `git push`. Push-ul este refuzat: *Updates were rejected because the remote contains work that you do not have locally*.
3. Maria aduce modificările lui Ion. Git anunță *CONFLICT (content): Merge conflict in …ReviewSection.jsx*:

   ```
   git pull --no-rebase
   ```

4. În editor, fișierul conține marcajele conflictului:

   ```
   <<<<<<< HEAD
         <h2>Ce spun cumpărătorii ({reviews.length})</h2>
   =======
         <h2>Părerile clienților ({reviews.length})</h2>
   >>>>>>> 4f9c2e1… (commit-ul lui Ion)
   ```

5. Între `<<<<<<< HEAD` și `=======` este varianta voastră, între `=======` și `>>>>>>>` cea venită de pe GitHub. Decideți împreună textul final. Fie apăsați în VS Code **Accept Current Change** / **Accept Incoming Change**, fie scrieți de mână rândul dorit și ștergeți toate cele trei marcaje. Salvați.
6. Maria încheie îmbinarea și publică rezultatul; apoi Ion rulează `git pull`:

```
git add frontend/app/src/components/ReviewSection.jsx
git commit -m "Rezolva conflictul din titlul recenziilor"
git push
```

Rezultat. `git log --oneline --graph -5` arată cele două commit-uri care se unesc într-un commit de îmbinare. Aplicația din browser afișează titlul ales, la amândoi.

6. Dovada contribuției fiecăruia

```
git log --oneline --author="Popescu"
git shortlog -sn
```

Prima comandă listează doar commit-urile lui Ion; a doua numără commit-urile fiecărui autor. Pe GitHub, alegeți branch-ul `proiect-popescu-ionescu` din lista de branch-uri și apăsați pe numărul de **commits**: lista arată autorul și data fiecărui commit și se poate filtra după autor. Pagina **Insights → Contributors** numără contribuțiile de pe branch-ul principal (`main`), deci pentru proiect folosiți lista de commit-uri a branch-ului comun [3].

7. Depanarea pas cu pas

Unde priviți	Ce vedeți acolo
DevTools → **Console** (F12)	erorile JavaScript (cu roșu), cu fișierul și rândul
DevTools → **Network** → cererea	**Headers**: adresa, metoda, codul de stare; **Payload**: ce s-a trimis; **Response**: ce a răspuns serverul
`docker compose -f backend/compose.yaml logs -f`	mesajele containerelor (nginx, PHP, PostgreSQL), în timp real; ieșiți cu **Ctrl + C**
`backend/app/var/log/dev.log`	jurnalul Symfony: fiecare cerere, interogare și excepție, cu clasa și rândul
Symfony Profiler	dacă este instalat, răspunsurile au antetul `X-Debug-Token-Link`; adresa lui deschide raportul complet al cererii [5]

7.1 Exercițiu: o eroare provocată și găsită

1. În `ReviewController.php`, metoda `byProduct()`, scrieți intenționat greșit `toArrray()` în loc de `toArray()`. Salvați.
2. Deschideți pagina unui produs. În locul recenziilor apare „Eroare 500”. Este simptomul, nu cauza.
3. **Network** → cererea `reviews` (roșie) → **Response**: pagina de eroare Symfony spune *Attempted to call an undefined method named "toArrray" of class "App\Entity\Review"*.
4. Confirmați în jurnal. Ultimele rânduri conțin `request.CRITICAL: Uncaught PHP Exception …`, cu numele fișierului și rândul:
   ```
   docker exec -it daw-php tail -n 20 var/log/dev.log
   ```
5. Corectați greșeala, salvați și reîncărcați pagina: cererea are din nou codul 200.

Sfat. Pentru orice eroare, mergeți în aceeași ordine: **simptomul** (ce vede utilizatorul) → **cererea** (Network: cod și răspuns) → **cauza** (jurnalul serverului). Citiți mesajul de eroare până la capăt: de cele mai multe ori spune exact fișierul și rândul.

8. Salvarea lucrului și oprirea

Verificați că ați corectat eroarea provocată, apoi, pe branch-ul comun (sau pe cel propriu):

```
git add .
git commit -m "Laboratorul 11: branch comun, conflict rezolvat"
git push origin proiect-popescu-ionescu
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Push. `git push origin proiect-popescu-ionescu` este forma completă; la proiectul individual, scrieți numele branch-ului vostru (`git push origin popescu.ion`). Forma scurtă, `git push`, merge doar dacă branch-ul local a fost legat anterior de cel de pe GitHub (`-u` face această legătură): la Ion, prin `git push -u origin proiect-popescu-ionescu`, în secțiunea 4; la Maria, automat, când a preluat branch-ul comun cu `git checkout`.

9. Exerciții

1. Împărțiți-vă în scris (de exemplu într-un fișier `PLAN.md` în repository) sarcinile proiectului pentru săptămânile rămase: cine face ce, pe ce fișiere.
2. Provocați voi o eroare de frontend (de exemplu o adresă greșită în `getReviews`) și găsiți-o doar cu DevTools. Ce cod de stare apare?
3. ★ (opțional) Repetați conflictul, dar rezolvați-l păstrând ambele modificări (**Accept Both Changes**), apoi explicați rezultatul.

10. Ce arătați la final

- branch-ul comun pe GitHub și lista lui de commit-uri, cu amândoi autorii;
- commit-ul de rezolvare a conflictului (`git log --oneline --graph`);
- rândul din `dev.log` care a indicat eroarea provocată și explicarea lui.

Resurse pentru exersare

- **W3Schools – Git branch merge (și conflicte):** https://www.w3schools.com/git/git_branch_merge.asp
- **Git, în limba română (Marius Ciurea, video):** https://www.youtube.com/watch?v=iVpuVkVeBcw
- **Pro Git, traducere în limba română:** https://git-scm.com/book/ro/v2
- **Chrome DevTools – Network:** https://developer.chrome.com/docs/devtools/network
- **Symfony – Logging:** https://symfony.com/doc/current/logging.html

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] S. Chacon și B. Straub, *Pro Git*, ed. a 2-a. Apress, 2014. Traducere în română: git-scm.com/book/ro/v2

[3] GitHub Docs, „Viewing a project’s contributors”. docs.github.com

[4] Chrome for Developers, „Inspect network activity”. developer.chrome.com

[5] Symfony, „Profiler” și „Logging”, documentație oficială. symfony.com/doc/current

[6] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*. Universitatea „Vasile Alecsandri” din Bacău, 2026.